ECE 4122/6122

Advanced Programming Techniques

Default Final Project: 3D Particle Simulation with OpenGL

Course	ECE 6122 — Advanced Programming Techniques
Assignment	Final Project
Points	300 points (30% of final grade)
Team Size	Individual submission (no group work)
Due Date	See Canvas — submitted as a single .zip archive

Project Overview

In this final project, you will design and implement a real-time 3D particle simulation using OpenGL, GLFW, and GLM, written entirely in C++17 (or later). The simulation will model a system of particles that interact with one another and with environmental forces such as gravity and boundary collisions. The project is designed to integrate the core concepts covered throughout the course: object-oriented design with custom classes, memory management, multithreading with OpenMP or std::thread, and GPU-accelerated rendering via the OpenGL pipeline.

The final deliverable will demonstrate mastery of modern C++ programming practices applied to a computationally demanding, visually engaging real-time system.

Learning Objectives

Upon completing this project, students will be able to:

- Design a multi-class C++ architecture that separates simulation logic from rendering logic
- Implement and manage OpenGL shader programs (vertex and fragment shaders written in GLSL)
- Apply rule-of-five semantics (constructor, destructor, copy/move constructor/assignment) to custom resource-owning classes
- Use parallel processing techniques (OpenMP or std::thread) to accelerate per-frame physics updates
- Employ smart pointers, STL containers, and modern C++ idioms throughout the codebase
- Demonstrate correct use of VAOs, VBOs, and dynamic buffer updates for real-time GPU data streaming

Required Class Design

Your submission must include at minimum the following custom C++ classes. You may add additional classes as you see fit.

1. Particle

Represents a single particle in the simulation. Must include:

- Member variables for position, velocity, acceleration, color, size (radius), mass, age, and lifetime
- A constructor that accepts initial state parameters

An `update(float dt)` method that advances the particle state by time step `dt` using a numerical integration scheme (Euler or Verlet)

- Appropriate getters/setters; mark read-only accessors const

A `isAlive()` method returning `bool` based on age vs. lifetime

2. ParticleEmitter

Responsible for spawning and managing a pool of Particle objects. Must include:

A configurable spawn rate (particles/second), emission cone angle, initial speed range, and color gradient stored as member data

A method `emit(float dt)` that creates new Particle objects at the correct rate and appends them to its internal container

A method `update(float dt)` that iterates over all active particles, calls their update methods, and removes dead particles

- Emitter configuration selectable at runtime (e.g., fountain, explosion)
- Internal storage via std::vector<Particle> or std::vector<std::unique_ptr<Particle>>

3. Renderer

Encapsulates all OpenGL state and draw calls. Must include:

- **RAII** ownership of at least one VAO, one VBO, and two shader programs (one for particles, one optional for the environment/floor)
- **RAII** stands for **Resource Acquisition Is Initialization** — a C++ idiom where resource lifetime is tied directly to object lifetime.

A `loadShaders(const std::string& vertPath, const std::string& fragPath)` method that compiles, links, and validates the shader program

A `draw(const std::vector<Particle>& particles, const glm::mat4& view, const glm::mat4& proj)` method that uploads particle data to the GPU and issues draw calls

- Point sprites or instanced rendering for efficient batch drawing of large particle counts (>= 5,000 particles)
- Proper cleanup of all OpenGL resources in the destructor

4. Camera

Implements an interactive first-person or arc-ball camera. Must include:

- Mouse-driven rotation and scroll-wheel zoom
- WASD keyboard movement (for first-person) or click-drag orbit (for arc-ball)

Methods `getViewMatrix()` and `getProjectionMatrix(float aspectRatio)` returning `glm::mat4`

5. SimulationConfig (struct or class)

A plain data container parsed from a JSON or INI configuration file at startup. Must expose at minimum:

- Window width and height
- Maximum number of particles
- Gravity vector (default: {0, -9.81, 0})
- Coefficient of restitution for boundary collisions
- Emitter type and initial emitter position

Physics & Simulation Requirements

The simulation must implement the following physical behaviors. You may extend beyond these minimums for extra credit (see Section 7).

1. **Gravity: Apply a constant gravitational acceleration vector to all particles each frame.**
2. **Boundary collisions: Particles must reflect off all six faces of a configurable axis-aligned bounding box. Apply the coefficient of restitution to the normal component of velocity upon collision.**
3. **Numerical integration: Implement at minimum first-order (Euler) integration. Semi-implicit Euler or Velocity Verlet are encouraged.**
4. **Parallel physics update: The per-particle physics update loop must be parallelized using OpenMP (#pragma omp parallel for) or std::thread with a thread pool. The number of threads must be configurable.**
5. **Particle aging and recycling: Each particle has a finite lifetime. Dead particles are recycled back into a free-pool rather than deallocated to avoid per-frame heap allocation.**

Rendering Requirements

The following OpenGL rendering features are mandatory. Shader source files must be stored as separate .vert and .frag files loaded at runtime.

- OpenGL 3.3 Core Profile (or higher) — no deprecated fixed-function pipeline calls
- At least two GLSL shader programs: one for the particle system, one for the bounding box wireframe
- Per-particle color interpolation in the fragment shader based on normalized particle age (young → old color gradient)
- Additive or alpha blending enabled for particle rendering to produce a glowing effect

- Point sprite rendering with gl_PointSize driven by distance from camera (perspective-correct sizing)
- Depth testing enabled; particles rendered after opaque geometry with depth writes optionally disabled

A rendered floor plane or bounding box wireframe to provide spatial reference

- Frame rate displayed in the window title bar using GLFW

Build & Submission Requirements

Build System

Your project must build cleanly using CMake (version 3.16 or later). A CMakeLists.txt must be provided at the project root. The build must succeed with the command sequence below on a standard Linux or Windows environment with GLFW, GLEW/GLAD, GLM, and OpenMP installed.

```
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
cmake --build . --config Release
```

Required Third-Party Libraries

Use only the following external libraries (all are available via vcpkg or standard package managers). You must not bundle pre-compiled binaries.

- GLFW 3.x — window creation and input
- GLEW — OpenGL function loading
- GLM — mathematics (vectors, matrices, quaternions)
- OpenMP — CPU parallelism (part of the compiler toolchain)
- nlohmann/json or inih — optional, for configuration file parsing

Directory Structure

Your submission ZIP must conform to the following layout:

```
LastName_FirstName_FinalProject/
  CMakeLists.txt
  README.md
  config/
    simulation.json
  shaders/
    particle.vert   particle.frag
    bbox.vert       bbox.frag
  src/
    main.cpp
    Particle.h / Particle.cpp
    ParticleEmitter.h / ParticleEmitter.cpp
```

```
Renderer.h / Renderer.cpp
Camera.h / Camera.cpp
SimulationConfig.h / SimulationConfig.cpp
```

Grading Rubric

Category	Points	Criteria
Class Design & OOP	20	All five required classes implemented; correct use of rule-of-five; appropriate const correctness; clear separation of concerns between simulation and rendering layers.
OpenGL Rendering	20	Core Profile pipeline; working vertex/fragment shaders loaded from files; correct VAO/VBO setup; per-particle color gradient; point sprites with perspective sizing; additive blending.
Physics Simulation	20	Gravity, boundary collision with restitution, and particle aging all function correctly; no tunneling artifacts at reasonable frame rates; numerical integrator is clearly identified.
Parallel Processing	15	OpenMP or std::thread parallelism applied to particle update; thread count configurable; no data races (verified by inspection or TSan); measurable speedup with large particle counts.
Memory Management	10	No memory leaks (verified by Valgrind or AddressSanitizer); particle recycling free-pool implemented; smart pointers used where ownership semantics require it.
Camera & Interactivity	5	Responsive mouse + keyboard camera control; perspective projection correct; no gimbal lock for arc-ball implementations.
Build System	5	CMakeLists.txt builds cleanly without modification; dependencies resolved automatically (vcpkg or find_package); Release config produces a working executable.
Code Quality	5	Consistent formatting; meaningful identifiers; no commented-out dead code; header guards or #pragma once; no compiler warnings at -Wall.
README	5	Clear build instructions; description of controls; list of implemented features; any known issues; screenshots or a video link encouraged.
Total	100	

Extra Credit Opportunities

The following enhancements may earn up to maximum of 10 additional points.

Feature	Points
CUDA kernel for particle physics update (replacing OpenMP)	**+5**
Particle-to-particle attraction/repulsion forces (e.g., gravitational or spring)	**+5**

Academic Integrity

This is an individual project. You may consult online documentation (cppreference.com, OpenGL documentation, GLFW/GLM documentation) and course materials freely. The following are prohibited:

- Sharing code with or receiving code from another student
- Using GitHub Copilot, ChatGPT, or any other AI code generation tool to produce significant portions of your submission
- Submitting code from a prior semester or from open-source particle system repositories as your own

All submissions will be checked for similarity. Any suspected violations will be referred to the Georgia Tech Office of Student Integrity. If you have a question about whether a particular resource is permissible, ask before using it.

Questions

Post all project questions to the course Piazza under the tag #FinalProject so that answers are visible to the whole class.